

CANN • ASCEND 950 • SIMD VF

Vector Function高性能编程指南 (二)

Broadcast / elemwise / reduce 范式：面向不同模式，提供高性能开发范式及原理讲解

KvRmsNormRopeCache：将基于 MemBase开发的算子迁移到 RegBase

技术解读 • CANN 社区 | 面向算子开发者

Contents

01

第一章

Broadcast · elemwise · reduce

常见范式的Regbase实现

02

第二章

外层数据流不动，把MemBase api 改为 RegBase VF

从 MemBase 切换到 RegBase

第一章

常见范式的Regbase实现

01

Broadcast

尾轴 / 首轴 / 中间轴

- 把长度为 1 的轴按逻辑重复到目标长度
- 把循环尽可能放在 VF 内
- 调整循环方向以提高性能

02

elemwise

throughput-bound

- 相邻迭代无循环携带依赖
- baseline 已最优，展开无收益
- 优化精力放在搬运 / 多核 / DoubleBuffer

03

reduce

latency-bound

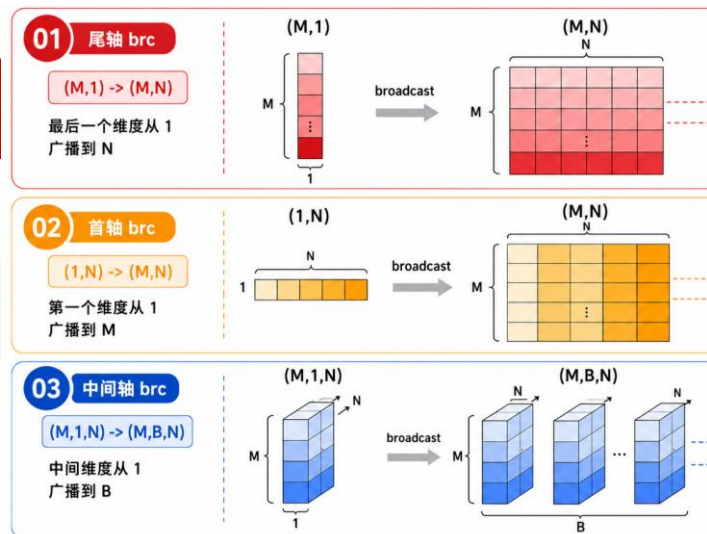
- 累加器带来循环间依赖
- 步数减不掉。多累加器藏延迟
- 可结合二分算法减少指令数

三种 Broadcast 模式

Broadcast: 把 Shape 中长度为 1 的轴, 重复到目标长度。

逻辑 Broadcast 的实现, 不需要真的复制出一个 Broadcast 后的 Tensor —— 在 VF 内按合适的循环顺序读原始数据, 在寄存器中完成逻辑广播, 再累加到目标位置即可。

模式名	Shape 变化模式	用例	broadcast 轴
尾轴 Broadcast	$(M, 1) \rightarrow (M, N)$	$y[m, n] = x[m, n] + a[m]$ // $M=17, N=653$	沿尾轴 N 重复
首轴 Broadcast	$(1, N) \rightarrow (M, N)$	$y[m, n] = x[m, n] + a[n]$ // $M=17, N=653$	沿首轴 M 重复
中间轴 Broadcast	$(M, 1, N) \rightarrow (M, B, N)$	$y[m, b, n] = x[m, b, n] + a[m, n]$ // $M=8, B=3, N=653$	沿中间轴 B 重复



所有更复杂的 Shape 均可以通过合轴简化为以上三种 Broadcast 模式其一

原始 Shape	合轴 Shape	应用模式
$(A1, A2, \dots, An, 1)$	$(A1*A2*\dots*An, 1)$	尾轴 Broadcast
$(1, A1, A2, \dots, An)$	$(1, A1*A2*\dots*An)$	首轴 Broadcast
$(H1, H2, \dots, Hm, 1, T1, \dots, Tn)$	$(H1*H2*\dots*Hm, 1, T1*\dots*Tn)$	中间轴 Broadcast

中间轴 Broadcast 可复用首轴 Broadcast 的计算逻辑

1. 固定一个外层 m
2. $x[m, :, :]$ 是 (B, N) 、 $a[m, :]$ 是 $(1, N)$
3. 计算 $y_m[b, n] = x_m[b, n] + a_m[n]$ — 正是首轴 Broadcast
4. For(m) 循环执行首轴 Broadcast

尾轴 Broadcast

用例 $M=17, N = 653$, 公式 $y[m, n] = x[m, n] + a[m]$, $a[m]$ 在逻辑上沿尾轴 n 重复, 等价于 $a[M]$ 看成 $a[M, 1] \rightarrow a[M, N]$ 。

优化实现: VF 内广播aReg到每行

```
// m 循环也收进一个 VF
for j in vfLoopM:
    remain=n
    LoadAlign<DIST_BRC_B32>(aReg, aAddr + j) // 广播读 a[j]
    for i in vfLoopN:
        mask = UpdateMask(remain);
        LoadAlign(xReg, xAddr + j*nPad + i*VL)
        Add(yReg, xReg, aReg, mask)
        StoreAlign(yAddr + j*nPad + i*VL, yReg, mask)

// kernel 侧: 单次 asc_vf_call, 不再逐行发起
```

性能对比

样例	VF counts	RVECEX counts	VF total cycles	PUSH_PB cycles
naive	17	374	1247	68
distbrc	1	374	356	8

结论: 1. 把行循环放进 VF
2. 把外部 Scalar 按行读 $a[j]$ 替换为 VF 内广播 $a[j]$ 到寄存器

原始实现: 按行读 $aLocal.GetValue(j)$, 每行发起 VF

```
__simd_vf__ void ComputeVF(xRowAddr, yRowAddr, n, vfLoopN, aRowValue) // 按行计算的 VF
// kernel 侧: 循环 M 轴分行计算
for j in M:
    asc_vf_call<ComputeVF>(xAddr + j*nPad, yAddr + j*nPad, n, vfLoopN, aLocal.GetValue(j));
```

首轴 Broadcast

用例 $M=17, N=653$, 公式 $y[m, n] = x[m, n] + a[n]$, $a[n]$ 沿首轴 m 重复。

优化点: 1. 把 m 循环放进 VF 后, 先按列遍历 n 复用 $aReg$; 2. 或先按行遍历 m , 再每行遍历 n 。

$n \rightarrow m$ 优化实现: 按列循环, 广播复用 $aReg$ 到每一行

```
for i in vfLoopN:
    mask = UpdateMask(n);
    LoadAlign(aReg, aAddr + i*VL) // 每个 n 块只读一次
    for j in vfLoopM:
        LoadAlign(xReg, xAddr + i*VL + j*nPad)
        Add(yReg, xReg, aReg, mask)
        StoreAlign(yAddr + i*VL + j*nPad, yReg, mask)
```

$m \rightarrow n$ 的自然写法按行 $\rightarrow$ 列循环, 也把 17 次 VF 合并成 1 次, 但每行重复读 $aReg$ 、重复更新 $mask$, 此时不如 $n \rightarrow m$ 。

```
for j in vfLoopM:
    remain = n
    for i in vfLoopN:
        mask = UpdateMask(remain);
        LoadAlign(aReg, aAddr + i*VL) // 每个 j 都重复读进 aReg
        LoadAlign(xReg, xAddr + i*VL + j*nPad)
        Add(yReg, xReg, aReg, mask)
        StoreAlign(yAddr + i*VL + j*nPad, yReg, mask)
```

性能对比

样例	VF counts	VF total cycles	RV_VLD	RV_PLT	循环方向
naive	17	839	374	187	粒度过碎
vfloop_nm	1	275	198	11	$n \rightarrow m$ 复用
vfloop_mn	1	442	374	187	$m \rightarrow n$ 连续

结论: 相比先行后列方向 ($m \rightarrow n$), 先列后行循环 ($n \rightarrow m$) 的内循环复用 $LoadAlign(a)$ 和 $UpdateMask(n)$, 显著减少了向量加载指令 VLD 和 Mask 生成指令 PLT。

中间轴 Broadcast

用例 M=8, B=3, N = 653, 公式 $y[m, b, n] = x[m, b, n] + a[m, n]$ 。

优化点：1. 把M和N轴都放进 VF；2. 确定循环的方向

性能对比

样例	VF counts	RVECEX counts	VF total cycles	RVECEX IPC	RV_VLD counts	RV_PLT counts	循环方向
naive	24	528	1168	0.45	528	264	逐 (m,b) 发起
vfloop_nb	8	352	884	0.40	352	88	m 次 · n→b
vfloop_bn	8	528	751	0.70	528	264	m 次 · b→n
vfloop_mbn	1	528	593	0.89	528	264	m→b→n 最快
vfloop_mnb	1	352	710	0.49	352	88	m→n→b

反直觉点

n->b 方向虽复用了 mask 和 aReg，但本例最内层循环数 B=3 太小，展开后无法喂饱指令并行度。省下的指令数不足以抵消损失的指令并行收益。

列循环优化（首轴 Broadcast）并非在全场景下都最优，需要具体 Shape 具体分析。

```
for m in M:
    for b in B:
        remain = n
        for i in N:
            mask = UpdateMask(remain)
            LoadAlign(aReg, aAddr + m*nPad + i*VL)
            LoadAlign(xReg, xAddr + (m*B+b)*nPad + i*VL)
            Add(yReg, xReg, aReg, mask)
            StoreAlign(yAddr + (m*B+b)*nPad, yReg, mask)
```

- 分阶段优化：
- 1. 初步实现，对 M*B 的每行，逐行做 a 的首轴 Broadcast: (1, N)->(M,N);
 - 2. 把 B、N 轴放进 VF，测出最优循环方向；
 - 3. 把 M 轴也放进 VF，根据 2. 的尝试确定可选循环方向，选择最优的。

三类 Broadcast 的优化经验：循环应融尽融进 VF

尾轴

$$y[m,n]=x[m,n]+a[m]$$

行标量 $a[m]$ 可以广播模式读入 $aReg$ ，复用在 x 一行内。

$17 \rightarrow 1$ VF

首轴

$$y[m,n]=x[m,n]+a[n]$$

先列后行的循环方向，可以复用加载的 a 块和生成的 $mask$ 。

$n \rightarrow m$ 最快

中间轴

$$y[m,b,n]=x[m,b,n]+a[m,n]$$

计算逻辑复用首轴 Broadcast 模式；优化方向应融尽融进 VF。

$m \rightarrow b \rightarrow n$ 最快

共同点：尽可能把循环融进 VF 内，能减少 `asc_vf_call` 次数与开销；把加载和计算都留在 VF 硬循环里，能提高指令的并行度。

逐元素 AXPY: 每个数独立

elemwise AXPY: $z = a \cdot x + y$, 元素间无依赖 (throughput-bound)

每个数独立计算, 没有循环间依赖 → 没有可缩短的串行依赖。



中间结果全程留在 RegTensor (不回写 UB)

三种写法 = 同一计算的不同「指令级并行」表达:

baseline	单组寄存器, 一轮 1 个 VL 块
unroll	4 组独立寄存器, 一轮 4 个 VL 块
pragma	循环体同 baseline, #pragma unroll 4 由编译器展开

每个数只读写自己的位置, 相邻循环之间没有循环间依赖, 向量单元每拍都能发射一条新指令——这类算子是 throughput-bound, 发射已经打满。下面先看 baseline 写法, 再看手写展开 / #pragma 能不能更快。

baseline：一轮算 1 个 VL 块，已打满向量发射

baseline：一轮算 1 个 VL 块，中间结果保存在 RegTensor

```
for i in loopNum:
    mask = UpdateMask(count)    // 开前 count 个 lane
    LoadAlign(vx, x + i*VL)    // 搬入无 mask
    LoadAlign(vy, y + i*VL)
    Muls(vt, vx, a, mask)      // vt = a * x
    Add(vt, vt, vy, mask)      // vt = a*x + y
    StoreAlign(z + i*VL, vt, mask) // 搬出带 mask
```

baseline 的流水数据

指标	值
PUSHQ VF cycles	625
VF IPC	1.47

每个数独立、相邻指令互不依赖、发射已打满，没有空泡可填、也没有串行依赖可以压缩，这是 throughput-bound 的典型特征。手写展开和 #pragma 能不能再快？

循环展开：手写 ×4 与 #pragma 都无收益

手写展开 ×4 (elemwise_axpy_unroll)

```
// 4 组独立命名寄存器，一轮算 4 个 VL 块
for (i = 0; i < loopNum; i += 4) {
    LoadAlign(vx0, x+(i+0)*VL); LoadAlign(vy0, y+(i+0)*VL);
    LoadAlign(vx1, x+(i+1)*VL); LoadAlign(vy1, y+(i+1)*VL);
    LoadAlign(vx2, x+(i+2)*VL); LoadAlign(vy2, y+(i+2)*VL);
    LoadAlign(vx3, x+(i+3)*VL); LoadAlign(vy3, y+(i+3)*VL);
    Muls(vt0, vx0, a); Add(vt0, vt0, vy0); // 第 0 路
    Muls(vt1, vx1, a); Add(vt1, vt1, vy1); // 第 1 路
    Muls(vt2, vx2, a); Add(vt2, vt2, vy2); // 第 2 路
    Muls(vt3, vx3, a); Add(vt3, vt3, vy3); // 第 3 路 (互不依赖)
    StoreAlign(z+(i+0)*VL, vt0); ... StoreAlign(z+(i+3)*VL, vt3);
}
```

#pragma unroll 4 (elemwise_axpy_pragma)

```
// 循环体与 baseline 完全相同，
// 只在 for 前加一行交给编译器展开
#pragma unroll 4
for (uint16_t i = 0; i < loopNum; i++) {
    mask = UpdateMask<float>(count);
    LoadAlign(vx, x + i*VL);
    LoadAlign(vy, y + i*VL);
    Muls(vt, vx, a, mask); Add(vt, vt, vy, mask);
    StoreAlign(z + i*VL, vt, mask);
}
```

两者等价：搬运与指令条数都≈baseline。手写只是把 4 个块的指令展开，#pragma 让编译器做同样的事。

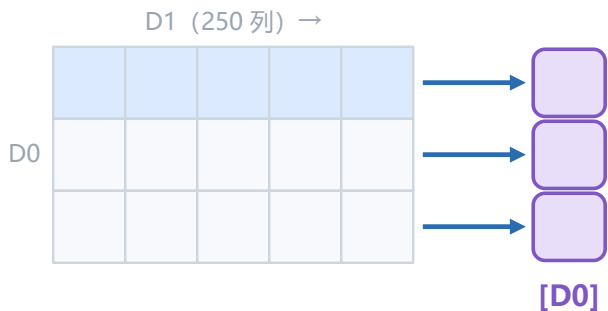
为什么无收益：elemwise 是 throughput-bound——相邻指令本就互不依赖、发射已打满；展开循环后，并没有获得更高并发度，反而增加了一些指令数。

归约是多对一：循环携带依赖 → latency-bound

两个归约轴：ar (axis=1, 行内) vs ra (axis=0, 跨行)

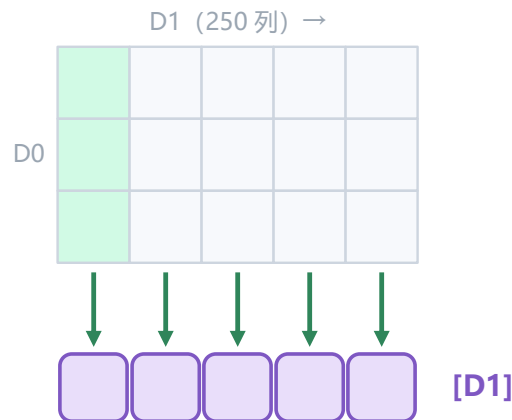
输入矩阵 [D0, D1] = [78 行, 250 列], 下用 3×5 网格示意; 紫色块 = 输出向量。

ar: 沿 D1 (行内) 归约 • 每行 → 1 标量 • 输出 [D0] (=78)



- 每行向量内 ReduceMax → 标量; StoreUnAlign 拼标量流
- 写法: baseline / unroll / pragma (binary 仅 reduce_sum)

ra: 沿 D0 (跨行) 归约 • 每列 → 1 值 • 输出 [D1] (=250)



- 跨行逐元素 Max (默认 ZEROING); 无向量内归约 → StoreAlign 整段
- 写法: baseline / unroll / pragma (无 binary)

步数减不掉

→ 多累加器手写展开藏延迟 (步数越多收益越大)

可结合 (求和)

→ 二分 / 树形折叠, 既减步数又减量, 通常最快

#pragma unroll

→ 对归约无效: 只复制循环体, 串行依赖还存在

串行依赖把延迟暴露成空泡：耗时 $\approx$ 串行步数 $\times$ 延迟 L

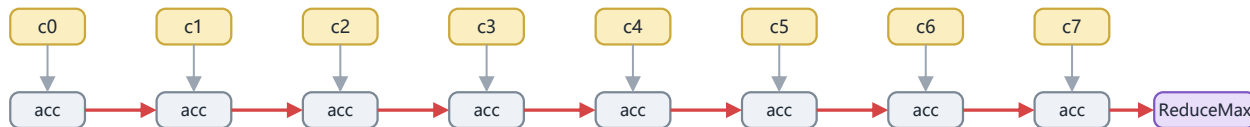
什么是串行步数

归约是 $\text{acc} = \text{op}(\text{acc}, x[i])$ ：要算下一条得先拿到上一条结果，只能一个接一个算。指令无法并行、首尾相接的指令有多少条，就是串行步数。

归约的两种组织方式（baseline 单路串行 vs unroll 多路并行，以 $N=8$ 为例）

红 = 最长串行路径（决定时延）。优化 = 缩短它。

① baseline 单路串行顺序折叠 串行步数 = $N = 8$



② unroll 多路并行 ($\times 4$) 4 路独立、最后合并 串行步数 $\approx N/4 + \log_2(4) = 4$

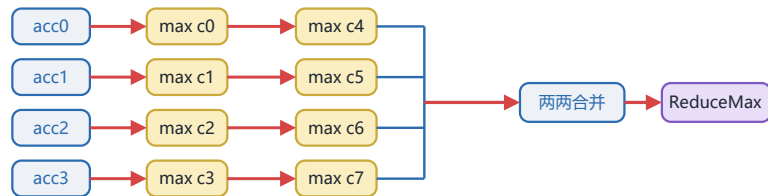


图 单路串行 vs 多路并行两种组织方式。

ar 与 ra 的基本写法：循环内外对调，搬出方式不同

ar (axis=1 沿尾维 · 每行出 1 个标量 · 输出 [D0])

```
for (i = 0; i < d0; i++) {           // 外层：行循环
    Duplicate(acc, init);             // NEG_INF (求最值)
    remainCols = d1;
    for (c = 0; c < rowChunks; c++) { // 内层：列块折进 acc
        mask = UpdateMask(remainCols);
        LoadAlign(in, x + i*d1Pad + c*VL);
        Max<MERGING>(acc, acc, in, mask); // 跨块必须 MERGING
    }
    ReduceMax(out, acc, fullMask);     // 向量内归约→标量
    StoreUnAlign(z, out, unalignAcc, 1); // 非对齐散出
}
StoreUnAlignPost(z, unalignAcc, 0);   // 冲刷尾部残留
```

ra (axis=0 沿首维 · 每列块出一整段向量 · 输出 [D1])

```
for (c = 0; c < colChunks; c++) {    // 外层：列块循环
    mask = UpdateMask(remainCols);    // 整列块共用，恒定
    Duplicate(acc, init);
    for (i = 0; i < d0; i++) {        // 内层：行折进 acc
        LoadAlign(in, x + i*d1Pad + c*VL);
        Max(acc, acc, in, mask);      // 第 i+1 条等第 i 条
    }
    StoreAlign(z + c*VL, acc, mask);   // 整段对齐搬出
}
// 无需 ReduceMax：每 lane 已是一列最值
// 无需散出：整段 64 lane 直接对齐写回
```

相同的shape [D0,D1]，归约轴变化，循环内外就对调。ar 每行 1 个标量 → 先 ReduceMax 成标量、再非对齐搬出 (StoreUnAlign 与 StoreUnAlignPost 成对)；ra 每列块产一整段向量 → mask 在行循环里恒定、整段对齐 StoreAlign。

reduce_max baseline: 单路串行, 指令省但空泡多

bad example: 单累加器 baseline (ra, 串行步数 78)

```
for c in colChunks:           // 外层: 列块
    Duplicate(accReg, NEG_INF)
    for i in d0 rows:         // 内层: 78 行单路串行
        LoadAlign(inReg, x + i*d1Pad + c*VL)
        Max(accReg, accReg, inReg, mask) // 第 i+1 条等第 i 条
    StoreAlign(z + c*VL, accReg, mask)
```

流水数据

指标	数值
PUSHQ VF cycles	510
VF IPC	0.63
RV_VMAX	312
MTE2 sum_cycles	5122

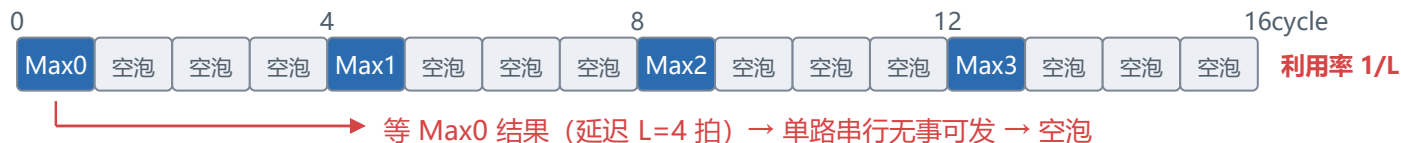
问题: RV_VMAX 只有 312 条、指令很少, 但 VF 仍要 510 cycle。原因是它们首尾相接, 下一条 Max 必须等上一条出结果, 时间都耗在等待上。

reduce_max 多路并行：4 路独立计算填满延迟空泡

为什么多路独立并行能提速：延迟隐藏 (latency hiding)

矢量指令「延迟 L 拍才出结果」，但矢量管线「每拍能发射 1 条」。下图每格 = 1 个发射拍 (cycle)，取 L=4。

① baseline 单路串行：下一条 Max 必须等上一条出结果



② unroll 多路并行 (×4)：4 路独立的 Max 轮流填满空泡



三条优化路径，本质都在「缩短或填满」这段串行依赖：

- unroll：指令数不变，用 4 路独立填满空泡（隐藏延迟）；还能配合硬件双发（每拍 2 条）。
- pragma：只复制循环体，仍串在同一 acc 上 → 串行步数不变、空泡照旧 → 对归约无效。

图 延迟隐藏：baseline 单路串行留空泡，unroll 4 路轮流填满。

reduce_max 的两种展开：手写多路展开有效，#pragma 无效

手动 4 次unroll (reduce_max_ra_unroll)

```
// 4 个独立最值变量, 4 路 Max 互不依赖
Duplicate(acc0..acc3, NEG_INF);
for (g = 0; g < d0/4; g++) {    // r = g*4
    LoadAlign(in0, x+(r+0)*d1Pad + c*VL);
    LoadAlign(in1, x+(r+1)*d1Pad + c*VL);
    LoadAlign(in2, x+(r+2)*d1Pad + c*VL);
    LoadAlign(in3, x+(r+3)*d1Pad + c*VL);
    Max(acc0,acc0,in0);    // 4 条 Max
    Max(acc1,acc1,in1);    // 互不依赖
    Max(acc2,acc2,in2);    // 填满延迟
    Max(acc3,acc3,in3);    // 空泡
}
Max(acc0,acc0,acc1); Max(acc2,acc2,acc3);
Max(acc0,acc0,acc2);    // 合并 4 路
```

#pragma unroll (无效)

```
// 只复制循环体, 没有解决串行的问题
Duplicate(accReg, NEG_INF);
#pragma unroll 4
for (i = 0; i < d0; i++) {    // 78 行
    LoadAlign(inReg, x + i*d1Pad + c*VL);
    Max(accReg, accReg, inReg); // 第 i+1
}                               // 仍等第 i
// 编译器不会擅自重排归约顺序
// (浮点不可结合) → 串行步数不变
```

串行步数决定提速幅度

轴	步数	base	unroll	#pragma
ra	78	510	358	516

- | 指令没减少，提速来自多路并发把空泡填上。
- | pragma 无效：Max 仍是串行的。

reduce_sum 的 ar 多累加器 unroll

每轮 4 行各一个累加器，跨块 MERGING

```
for g in groups:                // 每轮 4 行
    Duplicate(acc0..acc3, 0)
    for c in rowChunks:          // 4 行同列、共享 mask
        LoadAlign(in0..in3, ...)
        Add<MERGING>(acc0, acc0, in0, mask) // 末块 mask 不满
        Add<MERGING>(acc1..acc3, ...)      // 必须 MERGING
    ReduceSum + StoreUnAlign(acc0..acc3)
// 尾部 0~3 行单独收尾 + StoreUnAlignPost
```

收益

- | **ar 轴**: VF 792 → 670 (+15.4%)，与 reduce_max ar 同档。
- | **和 max 对对应**: 步数不减，只用 4 路独立 Add 减少指令发射延迟。
- | **为何不是最佳**: 浮点加法延迟更高、空泡更多；二分累加减步数与减量，远胜单纯展开。

对照 ra 轴: ra 步数 78, unroll 757 → 364 (+51.9%)
，为全部样例中最高。步数越多，多累加器回报越大。

reduce_sum 二分折叠：减串行步数 + 减量 + 提精度

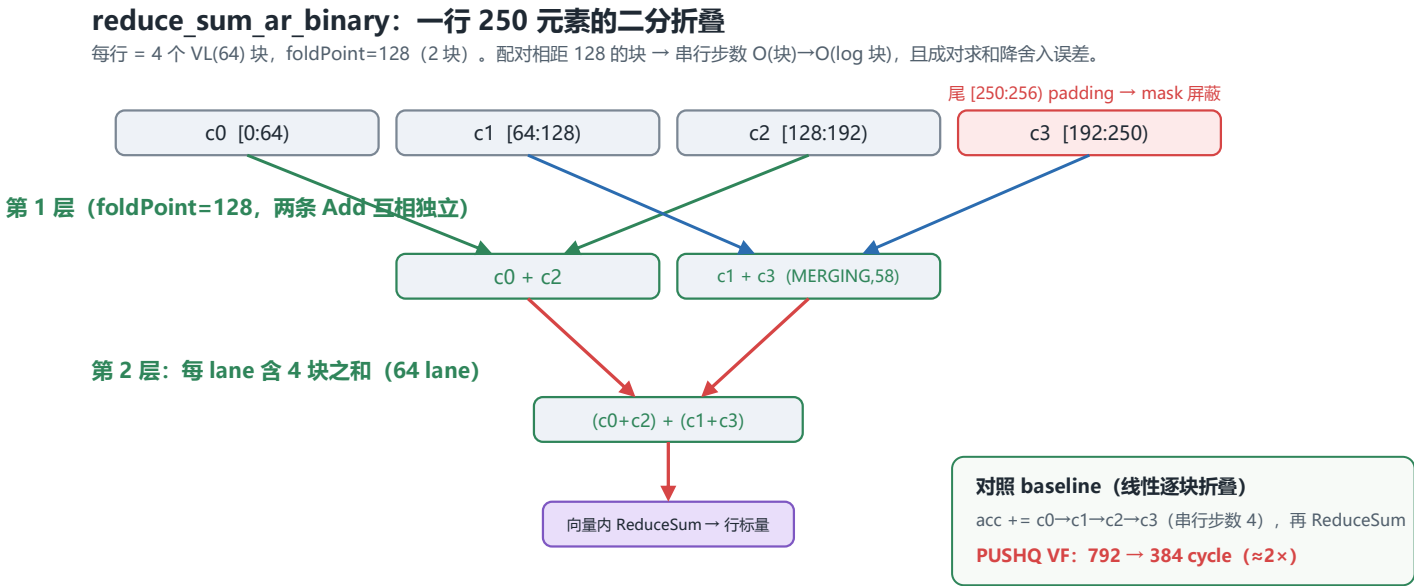


图 二分折叠：把 4 个列块顺序累加换成树形——(c0+c2) 与 (c1+c3) 互相独立、先各自算，再相加，串行步数从 4 缩到 2。

为什么只有 reduce_sum 用二分

二分把加法重新配对、换了求和顺序。浮点换序后结果只在低位有舍入差——reduce_sum 的校验是相对容差；而且成对求和本身就比较逐个累进的舍入更小。

reduce_max 选的是准确的最大元素、没有累积舍入，二分带来精度收益；它的串行步数靠多累加器降即可。

三种写法的区别

写法	VF 做了什么
baseline	792 逐块串行折叠
unroll	670 多累加器藏延迟
二分折叠	384 减步数 + 减量 + 精度提升

各轴各写法横向对比 (PUSHQ VF cycles)

范式	串行步数	baseline	unroll	pragma	二分折叠
reduce_max ar	4	699	592 (+15.3%)	699	—
reduce_max ra	78	510	358 (+29.8%)	524	—
reduce_sum ar	4	792	670 (+15.4%)	788	384 (+51.5%)
reduce_sum ra	78	757	364 (+51.9%)	720	—

结论

- reduce_max 多路并行最高 +29.8%。
- reduce_sum 多累加器 ra 最高 +51.9%。
- 二分 ar 约 2× (+51.5%)，是 ar 轴最快。
- 规律一致：串行步数越多，多累加器收益越大。

#pragma unroll 为何对归约无效

它只把循环体复制成多条，所有 Max / Add 仍是串行。编译器不会擅自重排归约顺序（浮点不可结合），串行步数不变、空泡照旧。要减步数或藏延迟，只能靠手写多累加器或二分。

归约范式的关键约定：MERGING、非对齐搬出

跨块折叠必须 MERGING

ar / axis=1 按块折叠时末块 mask 不满，默认 ZEROING 会把 mask 外清零、抹掉前面已折进的列。ra / axis=0 的 mask 在行循环里恒定，用默认 ZEROING 即可。

非对齐搬出 (ar)

每行只产生一个标量，用 StoreUnAlign 搬出，循环末尾使用 StoreUnAlignPost 刷新——二者必须成对

二维任意大小

LoadAlign 要 32B 对齐，第 i 行起点对任意 D1 未必对齐，CopyIn 逐行搬到按 32B 对齐的位置；行内按 VL(64) 分块，尾部不足由 mask 屏蔽。

第二章

从 MemBase 切换到 RegBase

在Ascend 950上可以使用MemBase编程方式实现算子，即Vector的单步计算都调用MemBase版本AscendC API

MemBase版本中Vector计算的每一步都会不断重复UB和寄存器之间的数据搬运

为发挥Ascend 950的最佳性能，需要把Vector计算重新使用Reg版本AscendC API改造

这里以 KvRmsNormRopeCache 的全载模板为例：保留已验证的数据流，只把 RMSNorm / RoPE 的计算下沉到 VF

1

算子计算过程

RMSNorm $\rightarrow$ v_cache, RoPE
 $\rightarrow$ k_cache

2

硬件与编程模型

GM / UB / Register 三层，区别只在 UB 内计算方式

3

MemBase 流程

LocalTensor 计算链，Vector 指令细碎

4

平滑迁移边界

外层结构复用，只替换compute body

5

VF 方案与理论

RMSNormVF / RopeVF，UB 临时张量大幅减少

6

RegBase 效果

同源 golden 校验，性能提升，Rope IPC $\approx$ 1.36

kv 输入尾轴拆两段：一段走 RMSNorm，一段走 RoPE

主要计算过程

```
kv[ ..., :Dv]  -> RMSNorm -> v_out -> v_cache  
kv[ ..., Dv:]  -> RoPE    -> k_out -> k_cache
```

RMSNorm (计算长, 主要优化点)

```
mean_square = mean(x * x)  
rms = sqrt(mean_square + epsilon)  
v_out = (x / rms) * gamma
```

含平方、归约、开方、除法、乘 gamma 与 BF16/FP32 转换，计算链较长。

RoPE (计算短)

```
real = rope_x[ ..., 0::2]  
imag = rope_x[ ..., 1::2]  
k_first  = real * cos_first  - imag * sin_first  
k_second = imag * cos_second + real * sin_second
```

复数旋转：从交错 rope 数据中拆出 real/imag，两段 cos/sin 完成旋转。

适合 VF：计算链短，用 VF 可避免 realFp32/imagFp32/y0/y1 等中间 UB 临时张量。

典型规格：当前 Dv=512、Dk=128。最后一维可全载。

三层存储模型：MemBase 与 RegBase 只差在 UB 内的计算方式

算子运行在三层存储 / 计算模型上

1. GM 层

输入输出与 k/v cache 位于全局内存

2. UB 层

DataCopy / DataCopyPad 把 tile 搬入本地 buffer

3. Register / VF 层

RegBase 在 `__simd_vf__` 内用 RegTensor 做寄存器级计算

面向 Ascend 950PR/950DT, NPU_ARCH=dav-3510; 固定 AIV Vector kernel, 数据类型 BF16。

迁移的核心：保留 GM 和 UB 之间搬运，只替换 UB 内计算

层级	MemBase 写法	RegBase 写法
GM/UB 搬运	DataCopyPad	保持相同
UB staging	LocalTensor / TQue	保持相同
计算对象	LocalTensor<T>	RegTensor<T>
计算入口	<code>__aicore__</code> 成员函数	<code>__simd_vf__</code>
UB↔寄存器	隐含在 Vector API	显式 LoadAlign/StoreAlign
tail 控制	count / repeat / mask	MaskReg

平滑迁移原则：保留已验证的数据流，只替换 Vector 计算部分。

Init → Process → ProcessTile: LocalTensor 计算, Vector 指令细碎

一次 tile 的数据流

```

Load gamma(BF16) -> Cast(FP32)
for each tile:
  CopyRopeAndX
    kv[..,Dv] -> xLocal
    kv[..,Dv:] -> ropeLocal
  Load cos/sin
  Rope    -> k outLocal
  CopyOutK -> k_out / k_cache
  RmsNorm -> v outLocal
  CopyOutV -> v_out / v_cache

```

RMSNorm 依赖多个 UB 临时区: xFp32 → square → rowSum → rms → outLocal。

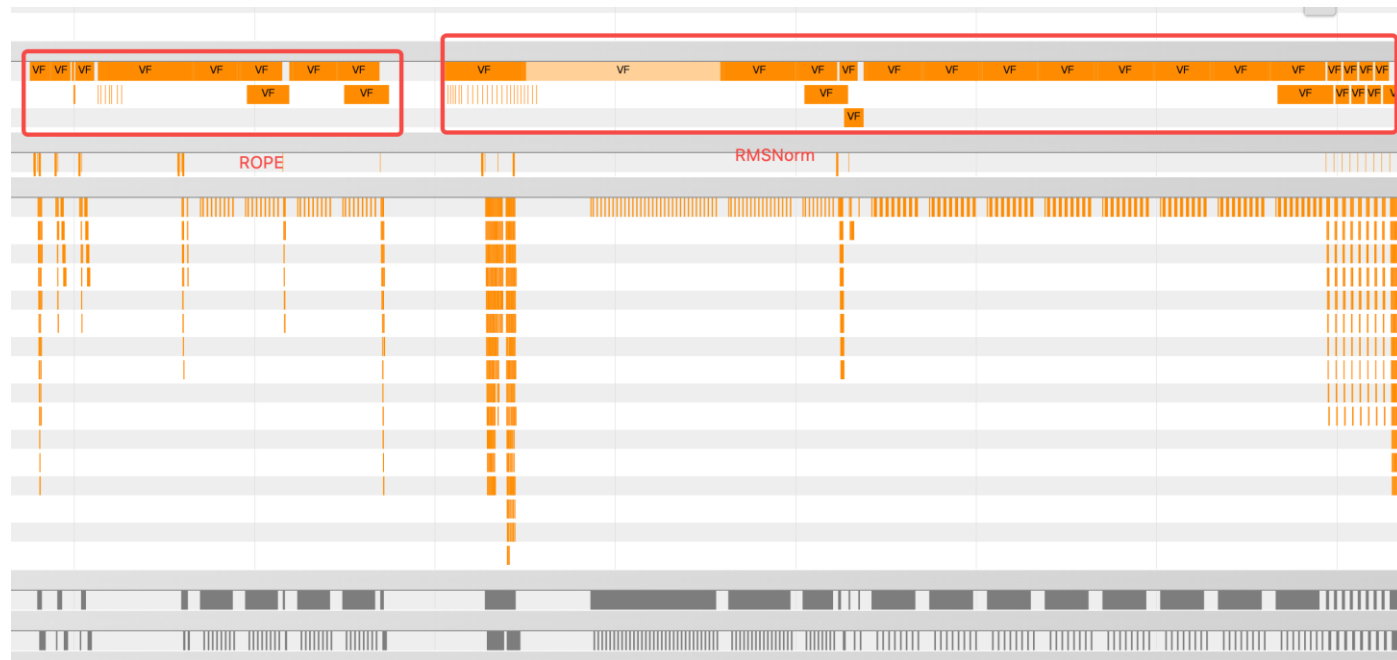


图 MemBase 仿真打点：左 RoPE、右 RMSNorm，每条 Vector 指令各自成一个 VF，整体细碎。

问题所在：标准 Vector API 之间通常形成 UB load/store 边界，中间结果反复落 UB，计算单元时间被搬运占据。

RoPE 与 RMSNorm 在 UB 临时张量间反复读写

RoPE: 以 LocalTensor 为核心

先将 cos / sin 转成 FP32。

GatherMask 从交错 rope 数据中抽取 real / imag。

两组乘加后 cast 回 BF16。

代价: 在 realFp32 / imagFp32 / y0 / y1 等 UB 临时张量间多次读写。

RMSNorm: 主要热点, 链路最长

```
cast B16->F32 -> square -> 分段累加  
-> WholeReduceSum -> *(1/Dv) -> +epsilon  
-> sqrt -> broadcast -> 逐行除法  
-> *gammaFp32 -> cast F32->B16
```

依赖多个 UB 临时区:

```
xFp32 -> square -> rowSum -> rms -> outLocal
```

小结: MemBase 写法直观、易验证, 但标准 Vector API 之间通常形成 UB load/store 边界, 中间结果反复落 UB——这正是 RegBase 把计算链下沉到寄存器后要消除的对象。

迁移不是推倒重写：外层结构复用，只替换 compute body

复用 MemBase 已验证的外层

- GM tensor 绑定—— Init 中地址绑定方式保持一致。
- Tiling 参数—— blockFactor / ubFactor / blockNum 一致。
- CopyIn / CopyOut—— DataCopyPad 搬运、cache 与 output 写回一致。
- Double buffer—— inQueue / cosSinQueue / outQueue 数据流一致。
- Golden 校验—— 同一套比较 k/v_cache、k/v_out。

真正变化：RmsNorm / Rope 的 compute body

```
// MemBase
LocalTensor -> Cast/Mul/Add/Reduce/Div -> LocalTensor

// RegBase
__ubuf__ ptr -> LoadAlign -> RegTensor
RegTensor Cast/Mul/Add/Reduce/Div
StoreAlign -> __ubuf__ ptr
```

这样划分边界的好处

- 外层索引、cache offset、GM 地址不变，迁移风险低。
- 原 golden 直接复用，MemBase 即 RegBase 的对照基线。
- 出错时可快速判断在 GM/UB 数据流还是 VF compute body。

RMSNormVF: 每行拆成两段 VF 循环, 中间量留在 RegTensor

第一段: 平方和

```
for i in Dv chunks:
    xB16    = LoadAlign(DIST_UNPACK_B16)
    xFp32   = Cast(BF16 -> FP32)
    square  = xFp32 * xFp32
    reduceSum += ReduceSum(square)
```

第二段: 归一化并乘 gamma

```
rms = sqrt(reduceSum / Dv + epsilon)
for i in Dv chunks:
    xFp32      = Cast(LoadAlign x)
    gammaFp32  = Cast(LoadAlign gamma)
    norm       = (xFp32 / rms) * gammaFp32
    StoreAlign(Cast(norm, FP32->BF16))
```

设计要点

■ **FP32 中间计算:** BF16 输入先升 FP32 做平方、归约、除法、乘法, 保证 RMSNorm 精度。

■ **寄存器驻留:** reduceSum / rmsValue / rmsBrc / norm 等短生命周期量留在 RegTensor。

■ **gamma 按需转换:** 不再预先 cast 成整段 FP32 UB 临时张量, 在 VF 中按 chunk load BF16 再 cast。

■ **tail 用 MaskReg:** UpdateMask<float>(remaining) 控制最后一个 chunk 的有效 lane, 避免 padding 当有效数据。

RopeVF: 从 BF16 直接拆 real/imag, 两段 cos/sin 完成复数旋转

核心代码

```
real, imag = LoadAlign(rope)
realFp32    = Cast Even
imagFp32    = Cast Odd

out_first   = realFp32 * cos_first
             - imagFp32 * sin_first
out_second  = imagFp32 * cos_second
             + realFp32 * sin_second

StoreAlign(out_first)
StoreAlign(out_second)
```

设计要点

| **全程寄存器**: real/imag、cos/sin、mul/result 都在寄存器内串联, 避免中间 UB 临时张量。

| **固定 mask**: pairCount = $Dk / 2 = 64$, mask 固定控制 64 个 FP32 lane。

| **两段写回**: 输出分前后两段, 与 golden 中 concat(real, imag) 的布局保持一致。

对照 MemBase: 省去 GatherMask 路径与 realFp32 / imagFp32 / y0 / y1 等多次 UB 读写。

UB 临时张量大幅减少；精度路径保持 BF16 → FP32 → BF16

UB 使用量减少

MemBase RMSNorm 需要多个 UB 临时张量：

```
xFp32 -> square -> rowSum -> rms -> outLocal
```

MemBase

gammaQueue: BF16 + FP32
wsBuffer_: rows×(Dv×3+Dk×8)×4B

RegBase

gammaQueue: BF16
无 wsBuffer_

- 减少中间结果落 UB，减少多段 Vector API 之间的 UB 往返。
- 中间状态尽量留在寄存器，只在 VF 入口 load、末尾 store。

精度路径保持一致

样例输入输出是 BF16，但关键中间计算必须用 FP32：

```
BF16 load -> FP32 compute -> BF16 store
```

- 通过 CAST_B16_TO_B32 / CAST_B32_TO_B16 明确表达转换路径。
- MemBase 与 RegBase 均通过同一套 golden 校验。

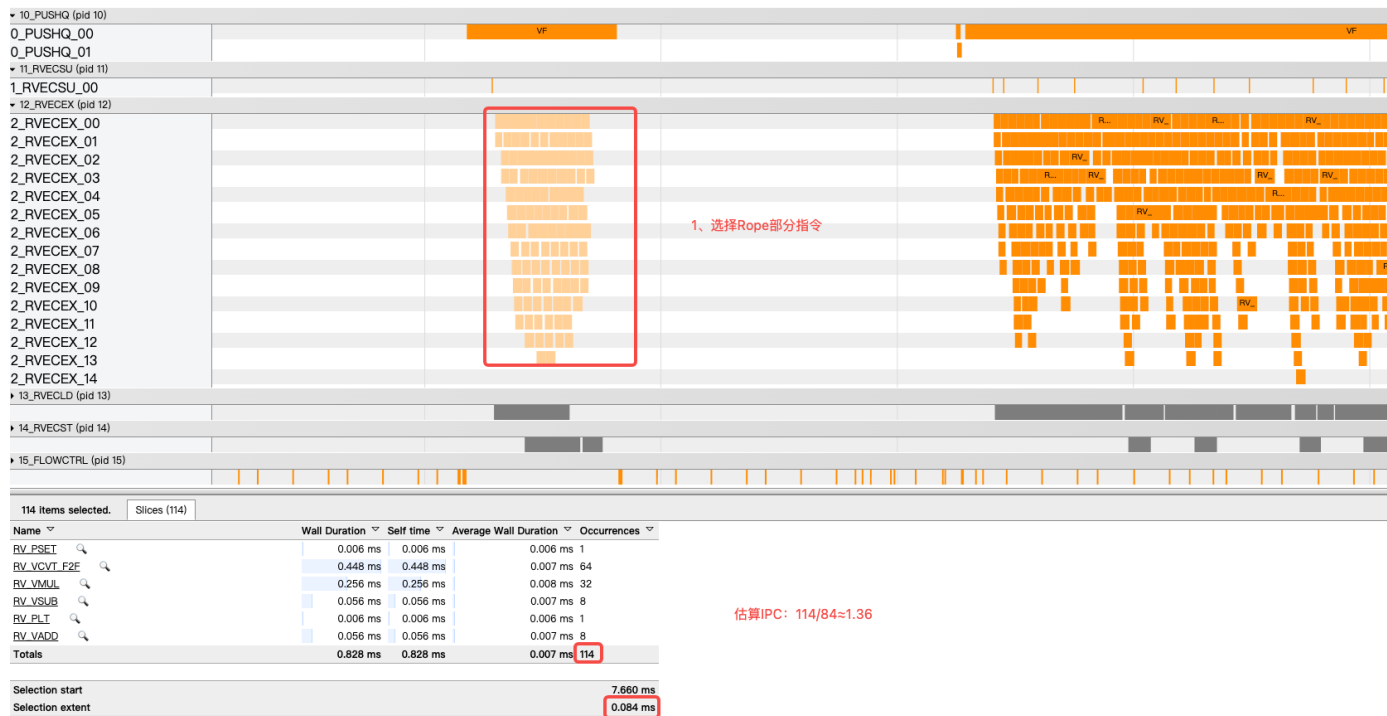
实测结果

MemBase: k_cache / v_cache / k_out / v_out 全部 PASS

RegBase: k_cache / v_cache / k_out / v_out 全部 PASS

RegBase v_cache / v_out 最大 diff = **0.0078125**，低于 BF16 比对阈值 **6e-2**。

Rope 与 RMSNorm 各融合成 1 个 VF; Rope IPC ≈ 1.36



Rope: 融合成 1 个 VF, RVEC_EX 估算 IPC ≈ 1.36 。

RMSNorm: 融合成 1 个 VF, IPC ≈ 1.15 。含单发指令, 且第一段循环体前后有依赖, IPC 比 Rope 低。

功能验证: membase / regbase full_load 均 PASS。

图 RegBase Rope 打点: 左侧 Rope 已融合成 1 个 VF, 选 RVEC_EX 区段估算 IPC ≈ 1.36 。

从 MemBase 切到 RegBase, 按这 8 步执行

1 保留 MemBase 的 host、tiling、queue、CopyIn、CopyOut。

2 找出 Vector 计算流程（本样例是 RMSNorm，其次 RoPE）。

3 把 Vector 计算流程拆成独立 __simd_vf__ 函数。

4 普通 __aicore__ 函数只做 GetPhyAddr() 到 __ubuf__ 指针的转换。

5 VF 内只用 RegTensor / MaskReg / Reg::Load* / Store* / compute API。

6 BF16 路径显式写清 BF16 → FP32 → BF16 的 cast trait 与 round mode。

7 数据 tail 用 MaskReg 控制。

8 每次优化后跑同一套 golden，确认 k/v_cache、k/v_out 全部 PASS。

总结

RegBase 的平滑迁移路线：外层不动、计算下沉、寄存器串联、同源校验。

Thank you.

社区愿景：打造开放易用、技术领先的 AI 算力新生态。

社区使命：使能开发者基于 CANN 社区自主研究创新，构筑共享共赢的 CANN 生态。

<https://gitcode.com/cann>



扫码报名

CANN开发者Meetup

6月27日 上海站